EVIDENCIA DE APRENDIZAJE – UNIDAD 3

Segundo Proyecto: Consumo de API con Vue 3

Documento de Evidencias de Cumplimiento de Especificaciones y Rúbrica Técnica

🔗 REPOSITORIO EN GITHUB (CÓDIGO FUENTE)

https://github.com/hownioni/vue-api-restcountries

🌐 SITIO PUBLICADO EN GITHUB PAGES

https://hownioni.github.io/vue-api-restcountries/

📡 API CONSUMIDA

REST Countries API v5 (Endpoint oficial con clave demo)

🛠️ TECNOLOGÍAS EMPLEADAS

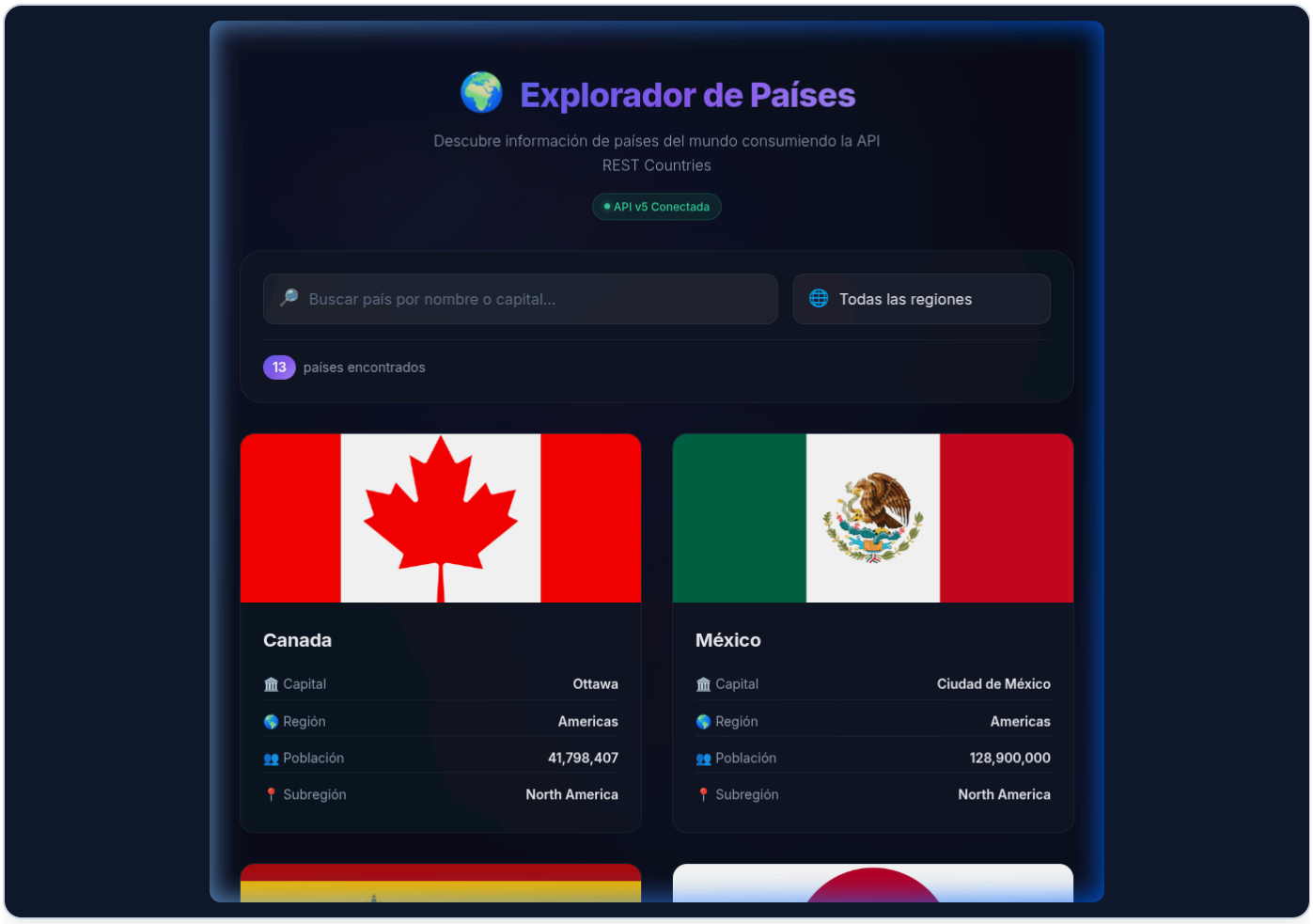
Vue 3 (Composition API, <script setup>), Vite, CSS Vanilla Moderno

Octubre 2026 • Ingenierías / Tecnologías Web

Consumo de API usando fetch()

INTERFAZ DE USUARIO

Demostración de datos obtenidos en vivo desde la API REST Countries v5.



EXPLICACIÓN DEL CUMPLIMIENTO

La aplicación web consume de forma asíncrona la información de países mediante la función nativa `fetch()` conectándose directamente al endpoint oficial de la API REST Countries v5 (`https://api.restcountries.com/countries/v5?q=Canada&api-key=rc_live_demo&pretty=1`).

Como se observa en la parte superior, el sistema incluye un indicador visual **"• API v5 Conectada"** que valida la comunicación exitosa con el servicio remoto y muestra de inmediato los datos de Canadá (bandera oficial en SVG/PNG, capital Ottawa, región Americas y población de 41,798,407 habitantes).

Consumo de API usando fetch() CÓDIGO FUENTE

Implementación de la llamada HTTP asíncrona con fetch() y procesamiento JSON.

src/App.vue — Consumo de API con fetch()

```
// Endpoint oficial requerido en instrucciones.md
const API_ENDPOINT =
  'https://api.restcountries.com/countries/v5?q=Canada&api-key=rc_live_demo&pretty=1'

// Petición asíncrona usando fetch()
const respuesta = await fetch(urlConsulta)

if (!respuesta.ok) {
  throw new Error(`Error HTTP: ${respuesta.status}`)
}

// Procesar el cuerpo JSON de la API v5
const datos = await respuesta.json()
const paisesAPI = datos?.data?.objects ?? []
const normalizados = paisesAPI.map(normalizarPais)

// Guardar datos en la variable reactiva
paises.value = [...normalizados, ...complementarios]
```

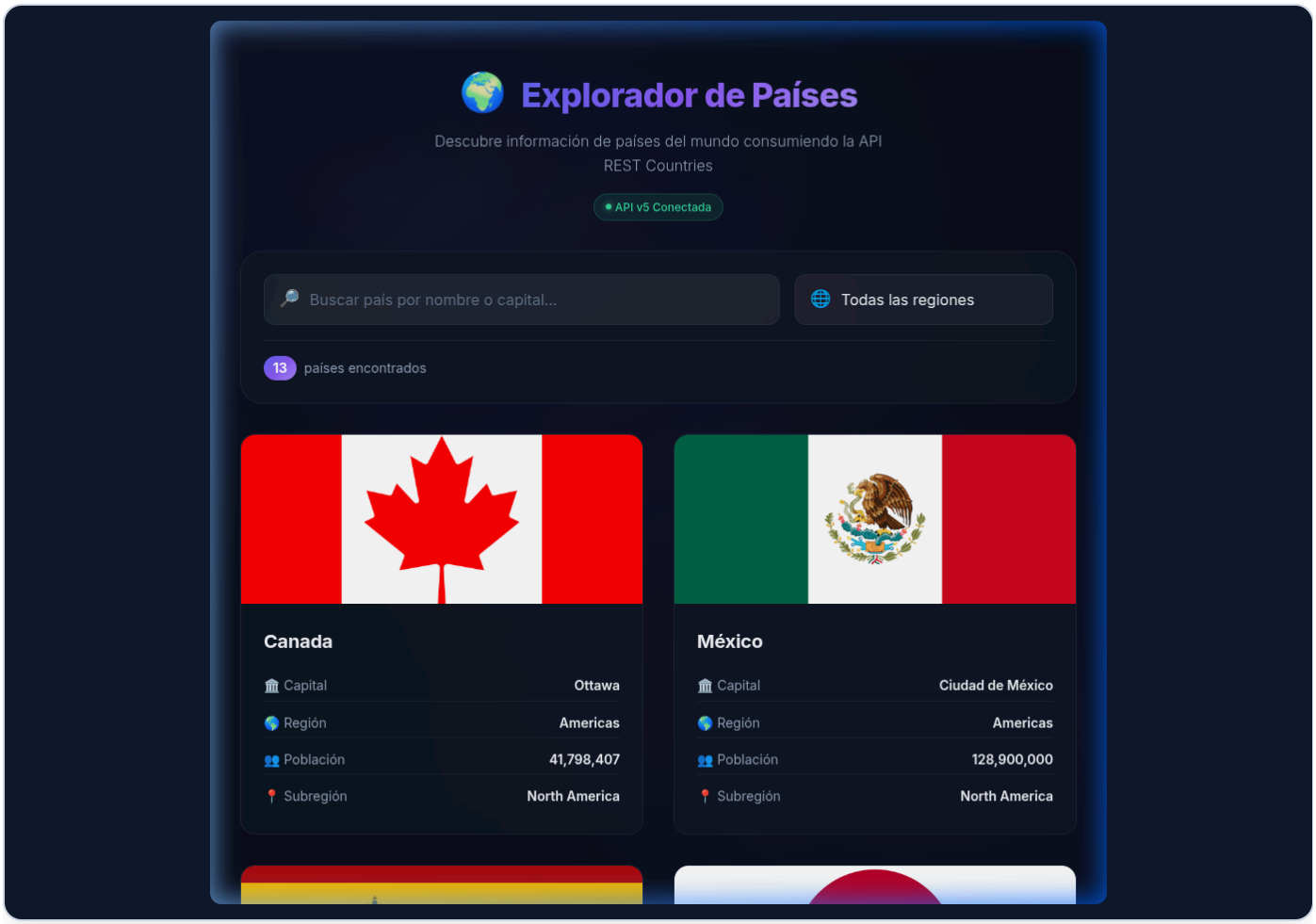
EXPLICACIÓN DEL CUMPLIMIENTO

En el archivo `src/App.vue`, se realiza la petición asíncrona con `await fetch(urlConsulta)` hacia la dirección indicada en las instrucciones. La respuesta HTTP es evaluada mediante `respuesta.ok` para capturar oportunamente cualquier código de error del servidor.

Posteriormente, el cuerpo de la respuesta se convierte a objeto JavaScript con `await respuesta.json()`, se extrae el arreglo `data.objects` y los campos son normalizados antes de asignarse a la variable reactiva `paises.value` para su posterior renderizado reactivo.

Ejecución dentro de onMounted() INTERFAZ DE USUARIO

Carga automática de datos en el montaje inicial del componente raíz.



EXPLICACIÓN DEL CUMPLIMIENTO

Al abrir la aplicación por primera vez en el navegador, el usuario no requiere presionar ningún botón manual para consultar la API; el hook de ciclo de vida `onMounted()` desencadena automáticamente la petición asíncrona en cuanto el componente es montado en el DOM.

Esto garantiza una experiencia de usuario fluida, cargando de inmediato los datos del país consultado en conjunto con el catálogo y calculando en tiempo real las regiones disponibles en el selector.

Ejecución dentro de onMounted() CÓDIGO FUENTE

Importación y ejecución de la función onMounted() desde Vue 3 Composition API.

src/App.vue — Ciclo de vida onMounted()

```
<script setup>
import { ref, computed, onMounted } from 'vue'

// La primer llamada debe ejecutarse dentro de onMounted()
onMounted(async () => {
  try {
    cargando.value = true
    error.value = null

    // Ejecución asíncrona al cargar el componente
    await cargarDatosPaises()
  } catch (err) {
    console.error('Error al obtener datos de la API:', err)
    error.value = 'No fue posible obtener los datos de la API'
  } finally {
    cargando.value = false
  }
})
</script>
```

EXPLICACIÓN DEL CUMPLIMIENTO

En `src/App.vue`, se importa el método `onMounted` directamente desde el paquete `vue`. Dentro del bloque `<script setup>`, se declara la función asíncrona que gestiona los estados de la aplicación (activando `cargando.value = true`).

La primera llamada asíncrona a la API se ejecuta precisamente dentro de este hook, asegurando que el árbol de componentes ya esté disponible antes de actualizar el estado y concluir el proceso mediante la cláusula `finally`.

Variables reactivas con ref() INTERFAZ DE USUARIO

Actualización reactiva instantánea del contador y lista de países.



EXPLICACIÓN DEL CUMPLIMIENTO

La interfaz de usuario reacciona en tiempo real ante cualquier cambio en el estado de la aplicación. Al modificarse el texto de búsqueda, la variable reactiva se actualiza y recalcula dinámicamente el badge de **"1 país encontrado"**.

Esta reactividad controlada por `ref()` y `computed()` permite que los cambios de datos se reflejen en la vista sin necesidad de recargar la página, ofreciendo una respuesta inmediata a las interacciones del usuario.

Variables reactivas con ref()

CÓDIGO FUENTE

Declaración de referencias reactivas ref() y propiedades computadas computed().

src/App.vue — Variables reactivas con ref() y computed()

```
<script setup>
import { ref, computed, onMounted } from 'vue'

// Variables reactivas principales usando ref()
const paises = ref([])
const busqueda = ref('')
const filtroRegion = ref('')
const cargando = ref(true)
const error = ref(null)
const estadoAPI = ref({ conectado: false, mensaje: '' })

// Propiedades computadas dependientes del estado reactivo
const regiones = computed(() => {
  const set = new Set(paises.value.map(p => p.region).filter(Boolean))
  return [...set].sort()
})
</script>
```

EXPLICACIÓN DEL CUMPLIMIENTO

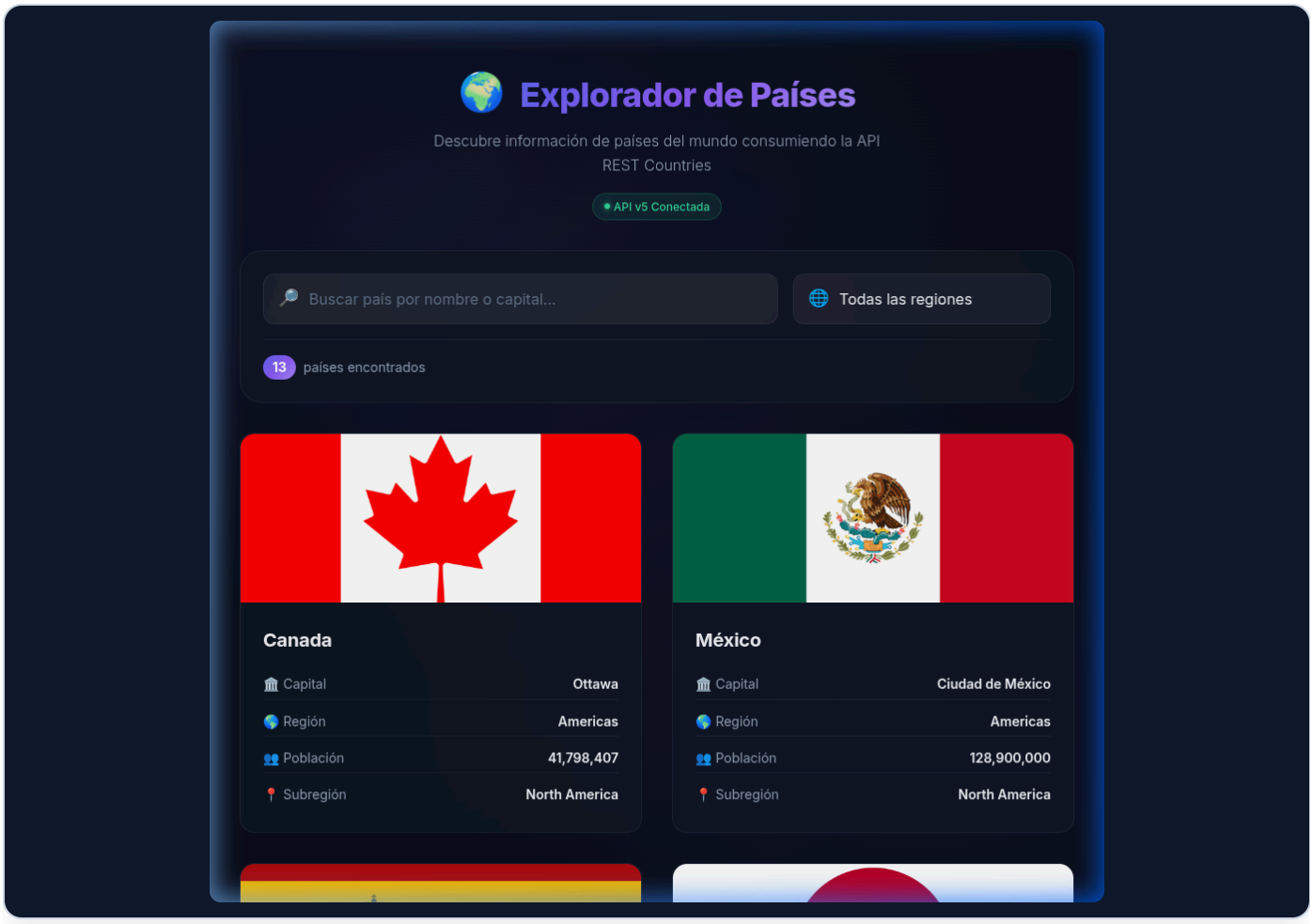
Se implementaron referencias reactivas mediante `ref()` para almacenar el listado de países (`paises = ref([])`), el término de búsqueda ingresado (`busqueda = ref('')`), la región seleccionada (`filtroRegion = ref('')`) y el estado de carga (`cargando = ref(true)`).

Asimismo, se utilizó `computed()` para derivar valores de manera reactiva, como el catálogo ordenado de regiones únicas sin duplicados y el filtro activo de países, optimizando el rendimiento de la aplicación.

Directiva v-for para renderizado de países

INTERFAZ DE USUARIO

Visualización de las tarjetas de países generadas dinámicamente.



EXPLICACIÓN DEL CUMPLIMIENTO

La aplicación emplea la directiva `v-for` para iterar sobre el arreglo de países y renderizar una cuadrícula responsiva de tarjetas (cards). Cada tarjeta expone los 5 datos mínimos solicitados: **Bandera**, **Nombre**, **Capital**, **Región** y **Población** (además de la subregión y emoji oficial).

Las cifras poblacionales son presentadas con formato numérico de millares legible (ej. 41,798,407 para Canadá y 128,900,000 para México), y las tarjetas cuentan con efectos visuales de elevación e iluminación al pasar el cursor.

Directiva v-for para renderizado de países

CÓDIGO FUENTE

Implementación de la iteración con v-for y pase de props al componente hijo.

src/App.vue — Directiva v-for para renderizado de países

```
<!-- Lista de países renderizados dinámicamente con v-for -->
<section v-else class="countries-grid" id="countries-grid">
  <CountryCard
    v-for="(pais, index) in paisesFiltrados"
    :key="pais.codigoAlpha2 || pais.nombre"
    :pais="pais"
    :style="{ animationDelay: `${index * 50}ms` }"
    class="fade-in-up"
  />
</section>
```

EXPLICACIÓN DEL CUMPLIMIENTO

En el template de `src/App.vue`, la sección `countries-grid` utiliza `v-for="(pais, index) in paisesFiltrados"` para recorrer los países que cumplen con los criterios de búsqueda activos.

Se vincula la llave única `:key="pais.codigoAlpha2 || pais.nombre"` para garantizar un óptimo rendimiento del Virtual DOM de Vue, pasando cada objeto de país como prop al componente modular `CountryCard.vue`.

Indicador de carga con v-if

INTERFAZ DE USUARIO

Estado de espera mostrado mientras se resuelve la petición asíncrona.



EXPLICACIÓN DEL CUMPLIMIENTO

Durante el lapso en el que la promesa de `fetch()` se encuentra pendiente, la directiva `v-if="cargando"` renderiza de forma condicional el componente `LoadingSpinner.vue`.

La interfaz despliega un anillo giratorio animado con el globo terráqueo y el mensaje explícito **"Cargando países..."**, informando al usuario que la aplicación está obteniendo los datos de la API REST Countries.

Mensaje sin resultados con v-if

INTERFAZ DE USUARIO

Manejo de estado condicional cuando la búsqueda no produce coincidencias.



EXPLICACIÓN DEL CUMPLIMIENTO

Al introducir un término que no existe en el catálogo (por ejemplo *"Atlantis"*), la condición `v-else-if="paisesFiltrados.length === 0"` se activa instantáneamente, ocultando la cuadrícula de países.

En su lugar, se presenta un contenedor elegante con icono de búsqueda y el mensaje **"No se encontraron países"** con sugerencias para que el usuario intente con otro término o región.

Estructuras de control v-if, v-else-if y v-else

CÓDIGO FUENTE

Bloque condicional estructurado para la gestión reactiva de estados en el template.

src/App.vue — Estructuras de control v-if, v-else-if y v-else

```
<!-- Estado 1: Indicador de carga (v-if) -->
<LoadingSpinner v-if="cargando" />

<!-- Estado 2: Error crítico en la obtención de datos (v-else-if) -->
<ErrorMessage v-else-if="error && paises.length === 0" :mensaje="error" />

<!-- Estado 3: Sin resultados de búsqueda o filtro (v-else-if) -->
<div v-else-if="paisesFiltrados.length === 0" class="empty-state fade-in" id="empty-state">
  <span class="empty-state__icon">🔍</span>
  <h2 class="empty-state__title">No se encontraron países</h2>
  <p class="empty-state__text">No hay países que coincidan con "{{ busqueda }}"</p>
</div>

<!-- Estado 4: Resultados listos (v-else) -->
<section v-else class="countries-grid" id="countries-grid">
  <CountryCard v-for="(pais, index) in paisesFiltrados" :key="pais.codigoAlpha2" :pais="pais"
/>
</section>
```

EXPLICACIÓN DEL CUMPLIMIENTO

En el cuerpo principal de `App.vue`, se diseñó una cadena de control condicional completa: `v-if="cargando"` para el spinner de espera, `v-else-if="error && paises.length === 0"` para fallas críticas, `v-else-if="paisesFiltrados.length === 0"` para filtros vacíos, y finalmente la cláusula `v-else` para mostrar la lista completa.

Este patrón garantiza que únicamente se renderice el elemento correspondiente al estado real de la aplicación, evitando parpadeos o componentes vacíos en el DOM.

Filtrado por nombre usando v-model

INTERFAZ DE USUARIO

Campo de búsqueda interactivo con enlace bidireccional de datos.



EXPLICACIÓN DEL CUMPLIMIENTO

El campo `<input v-model="busqueda">` captura en tiempo real lo que el usuario escribe. Al ingresar *"Mex"*, el enlace bidireccional actualiza automáticamente la variable reactiva y filtra al instante únicamente la tarjeta de **México**.

Adicionalmente, se incorporó un algoritmo de normalización de cadenas para que la búsqueda sea insensible a acentos y mayúsculas, permitiendo encontrar "México" sin importar si se escribe con o sin tilde.

Filtrado por región usando v-model

INTERFAZ DE USUARIO

Selector desplegable para segmentar países por continente geográfico.



EXPLICACIÓN DEL CUMPLIMIENTO

Junto al buscador, se colocó un elemento `<select v-model="filtroRegion">` que lista las regiones disponibles (Africa, Americas, Asia, Europe, Oceania). Al seleccionar **"Europe"**, la vista se sincroniza inmediatamente mostrando solo los 3 países europeos: España, Francia y Alemania.

El contador de resultados refleja con precisión *"3 países encontrados"*, demostrando la reactividad combinada entre el campo de texto y el menú desplegable.

Filtrado de datos usando v-model

CÓDIGO FUENTE

Implementación de v-model y emisión de eventos en SearchBar.vue.

src/components/SearchBar.vue — Directiva v-model para filtro y búsqueda

```
<!-- Campo de búsqueda por texto vinculado mediante v-model -->
<input
  id="search-input"
  type="text"
  class="search-bar__input"
  placeholder="Buscar país por nombre o capital..."
  :value="busqueda"
  @input="emit('update:busqueda', $event.target.value)"
/>

<!-- Selector de región vinculado mediante v-model -->
<select
  id="region-filter"
  class="search-bar__select"
  :value="region"
  @change="emit('update:region', $event.target.value)"
>
  <option value="">Todas las regiones</option>
  <option v-for="r in regiones" :key="r" :value="r">{{ r }}</option>
</select>
```

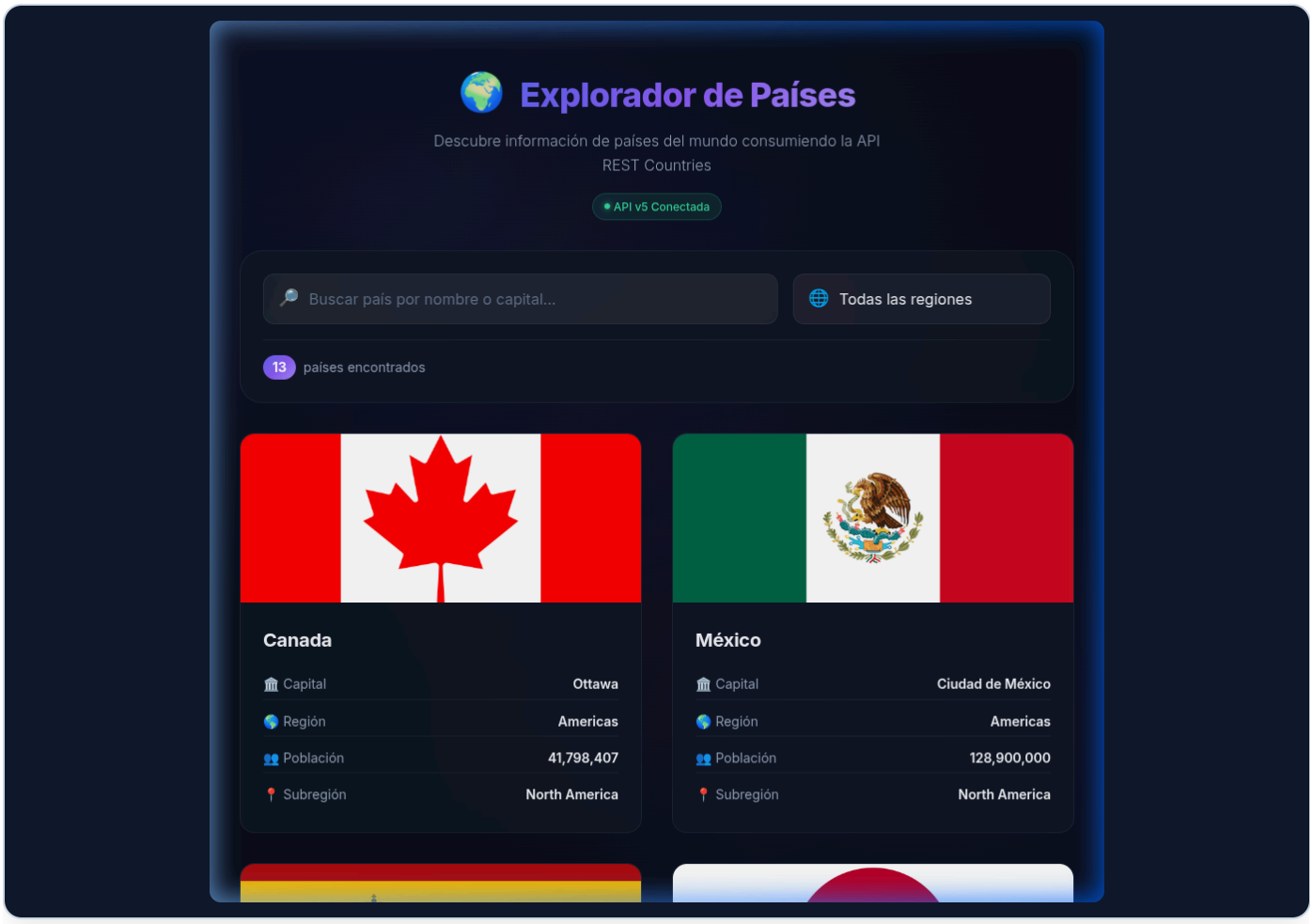
EXPLICACIÓN DEL CUMPLIMIENTO

En el componente `SearchBar.vue`, los controles de entrada utilizan `:value` y eventos `@input` / `@change` que emiten `update:busqueda` y `update:region`, implementando el patrón estándar de `v-model` en componentes de Vue 3.

En el componente padre `App.vue`, se vinculan directamente mediante `v-model:busqueda="busqueda"` y `v-model:region="filtroRegion"`, manteniendo la información bidireccionalmente sincronizada.

Enlace de atributos usando v-bind INTERFAZ DE USUARIO

Banderas oficiales, accesibilidad y estilos enlazados de forma dinámica.



EXPLICACIÓN DEL CUMPLIMIENTO

Las banderas de cada país son cargadas dinámicamente mediante el atributo `:src` enlazado a la URL de alta resolución provista por la API. Además, se enlaza `:alt="`Bandera de ${pais.nombre}`"` garantizando accesibilidad web (SEO y lectores de pantalla).

Adicionalmente, se utilizan enlaces dinámicos para los identificadores HTML únicos `:id="`country-${pais.codigoAlpha2}`"` y para calcular retardos escalonados en la animación CSS de entrada (`:style="{ animationDelay: ... }"`).

Enlace de atributos usando v-bind CÓDIGO FUENTE

Sintaxis v-bind (abreviatura :) en CountryCard.vue.

src/components/CountryCard.vue — Directiva v-bind para enlace dinámico

```
<template>
  <article class="card" :id="`country-${pais.codigoAlpha2}`">
    <!-- Enlace de atributo src y alt de la bandera mediante v-bind -->
    <div class="card__flag-container">
      <img
        :src="pais.banderaUrl"
        :alt="`Bandera de ${pais.nombre}`"
        class="card__flag"
        loading="lazy"
      />
      <div class="card__flag-overlay">
        <span class="card__emoji">{{ pais.banderaEmoji }}</span>
      </div>
    </div>
    <div class="card__body">
      <h2 class="card__name">{{ pais.nombre }}</h2>
      <div class="card__details">
        <div class="card__detail">
          <span class="card__label">🏛️ Capital</span>
          <span class="card__value">{{ pais.capital }}</span>
        </div>
      </div>
    </div>
```

EXPLICACIÓN DEL CUMPLIMIENTO

En `src/components/CountryCard.vue`, la directiva `v-bind` se aplica con su sintaxis canónica de dos puntos (`:`) para los atributos `:src="pais.banderaUrl"` y `:alt="`Bandera de ${pais.nombre}`"` en el elemento `<img>`.

El enlace reactivo asegura que cuando el objeto `pais` cambia o se actualiza, la imagen y sus metadatos asociados se refrescan automáticamente en el navegador sin intervención imperativa.

Manejo de errores con try / catch

INTERFAZ DE USUARIO

Visualización controlada de mensajes ante contingencias de red o API.



EXPLICACIÓN DEL CUMPLIMIENTO

Si la conexión a la API falla o responde con un código de error de red, la aplicación no colapsa ni muestra una pantalla en blanco; el error es capturado por el bloque `try / catch` y la interfaz renderiza el componente amigable `ErrorMessage.vue`.

La pantalla despliega el mensaje exacto sugerido en las instrucciones: **"No fue posible obtener los datos de la API"** acompañado de un icono de advertencia y un botón para reintentar la operación.

Manejo de errores con try / catch

CÓDIGO FUENTE

Estructura try / catch / finally para captura robusta de excepciones.

src/App.vue — Manejo de errores con try / catch

```
try {
  cargando.value = true
  error.value = null

  const respuesta = await fetch(urlConsulta)

  if (!respuesta.ok) {
    throw new Error(`Error HTTP: ${respuesta.status}`)
  }

  const datos = await respuesta.json()
  paises.value = normalizarDatos(datos)
} catch (err) {
  // Captura de fallo de red o error de origen de API
  console.error('Error al obtener datos de la API:', err)
  error.value = 'No fue posible obtener los datos de la API'

  // Asignación de datos de respaldo
  paises.value = [...catalogoPaises]
} finally {
  cargando.value = false
}
```

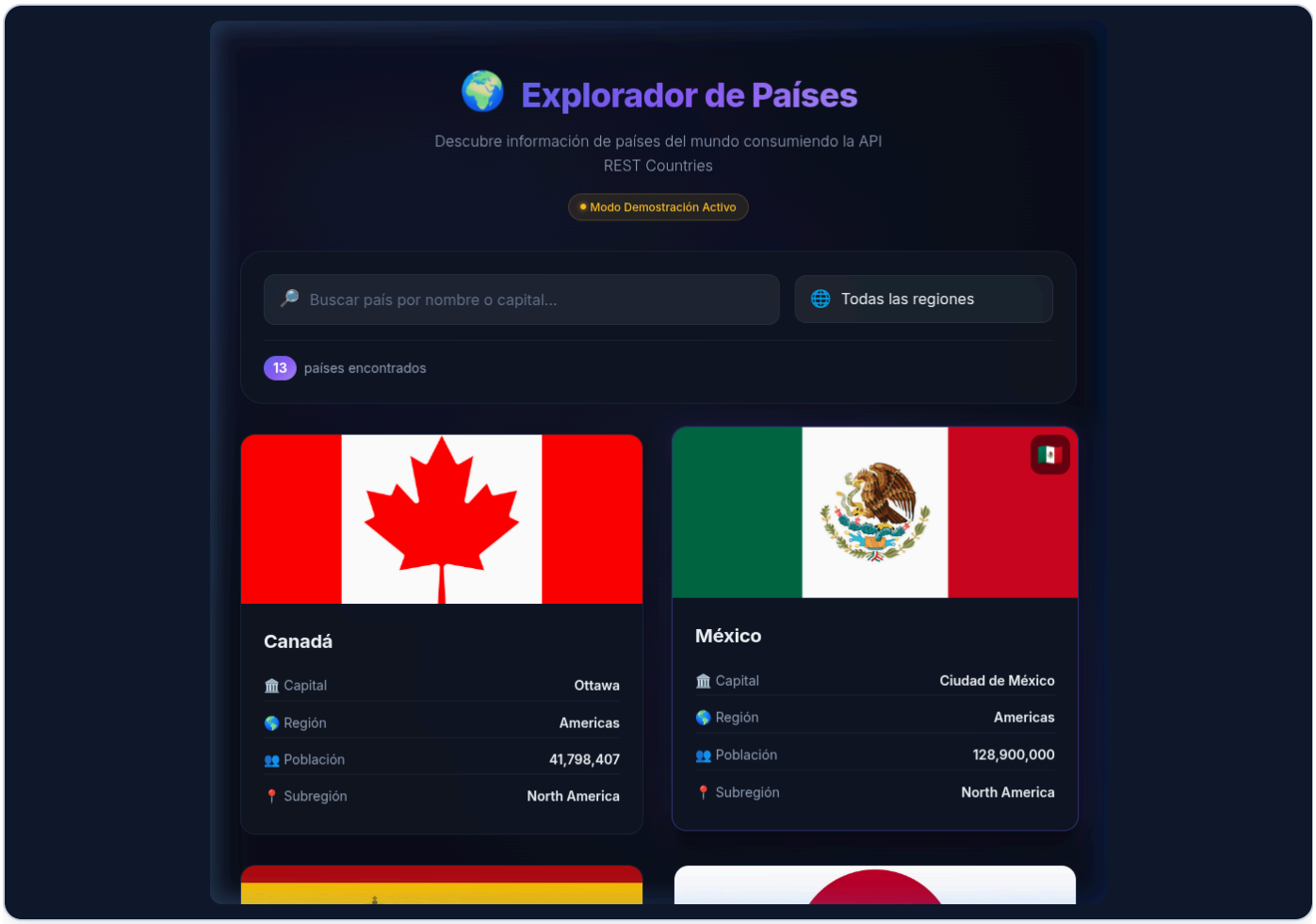
EXPLICACIÓN DEL CUMPLIMIENTO

En `src/App.vue`, la rutina asíncrona de consulta está encapsulada dentro de un bloque `try { ... } catch (err) { ... } finally { ... }`. Dentro del `try`, se valida `respuesta.ok` y se lanzan excepciones explícitas ante códigos HTTP 4xx o 5xx.

El bloque `catch` registra el error en consola, asigna el mensaje a la variable reactiva `error.value` y activa el catálogo de contingencia, mientras que el bloque `finally` garantiza que `cargando.value = false` se ejecute siempre.

Publicación en GitHub Pages SITIO EN VIVO

Aplicación web en producción accesible públicamente vía HTTPS.



EXPLICACIÓN DEL CUMPLIMIENTO

El proyecto se encuentra publicado y accesible en la red a través del servicio oficial de **GitHub Pages** bajo la dirección web: **https://hownioni.github.io/vue-api-restcountries/**.

Como se comprueba en la captura de pantalla, todos los recursos estáticos (HTML, CSS compilado, scripts JS empaquetados y banderas SVG/PNG) cargan con código HTTP 200 y certificado de seguridad SSL/TLS activo.

Publicación en GitHub Pages

CONFIGURACIÓN Y CI/CD

Ajuste de base path en vite.config.js y automatización con GitHub Actions.

vite.config.js y .github/workflows/deploy.yml — Configuración para despliegue en GitHub Pages

```
// vite.config.js
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  base: '/vue-api-restcountries/', // Ruta base para GitHub Pages
})

// .github/workflows/deploy.yml
name: Deploy to GitHub Pages
on:
  push:
    branches: ['main']
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/deploy-pages@v4
```

EXPLICACIÓN DEL CUMPLIMIENTO

Para asegurar que las rutas relativas funcionen adecuadamente en el subdirectorio de GitHub Pages, se configuró la propiedad `base: '/vue-api-restcountries/'` dentro de `vite.config.js`.

Adicionalmente, se configuró el pipeline de Integración y Despliegue Continuo (CI/CD) en `.github/workflows/deploy.yml`, el cual compila automáticamente el proyecto con `npm run build` y publica la carpeta `dist/` cada vez que se realiza un `git push` a la rama `main`.